Numerical Computing with NumPy

Prepared By

Dr. Islam Zakaria

Lecturer at the Information Technology Department, FCAIH, Capital University

Why We Need NumPy

The Importance of Numpy

1. **Performance**
   - **NumPy uses optimized C and Fortran libraries**, enabling operations that are **10–100× faster** than pure Python loops.
   - Designed for **vectorized operations**, processing whole arrays at once instead of element-by-element.
2. **Memory Efficiency**
   - NumPy arrays store data in **contiguous memory blocks**, unlike Python lists.
   - **Enables fast access, compact storage, and efficient multi-dimensional structures**.

The Importance of Numpy

3. **Provides a rich set of tools for:**
   - **Linear algebra**
   - **Broadcasting**
   - **Random number generation**
   - **Fourier transforms**
   - **Masking and filtering**
   - **Statistical operations**

The Importance of Numpy

4. **Foundation of the Scientific Python Ecosystem**

   NumPy is the core dependency for libraries like:

   - **SciPy**
   - **Pandas**
   - **Matplot**
   - **libscikit-learn**
   - **TensorFlow & PyTorch**

   Without NumPy's array model, modern scientific computing in Python would not exist.

5. **Interoperability & Standardization**

   - NumPy arrays are the standard data format in data science and machine learning.
   - Efficiently interface with C/C++, GPUs, and compiled libraries.

Foundations, Architecture, and Performance

Defining Numerical Computing

- In Computer Science, we often distinguish between "Symbolic Computing" (manipulating symbols and algebra exactly) and "Numerical Computing."
- **Definition:** Numerical computing involves algorithms for solving problems of continuous mathematics. It deals with approximations, floating-point arithmetic, and iterative solutions rather than exact symbolic resolution.
- **The Difference:**
  - **Discrete/Text Algorithms:** Sorting a list of strings, traversing a graph, searching text. These are exact.
  - **Numerical Algorithms:** Solving differential equations, matrix multiplications, optimization. These deal with **continuous variables** and **error propagation**.

The Python Data Science Stack

- We must contextualize NumPy not as a standalone library, but as the foundational bedrock of the entire ecosystem.
  - **Layer 1 (The Host):** Python. The general-purpose glue logic. It handles file I/O, web requests, and high-level logic.
  - **Layer 2 (The Foundation): NumPy**. It provides the ndarray (N-dimensional array). It is the standard for exchanging data between algorithms. Crucially, libraries like **OpenCV** (computer vision) and **PyTorch** often allow zero-copy interchange with NumPy.
  - **Layer 3 (The Tools): SciPy** (Advanced math/physics algorithms), **Matplotlib** (Visualization), **Pandas** (Tabular data/Statistics).

The Debate: Python vs. MATLAB

- For decades, MATLAB was the king of engineering. **Why has Python/NumPy displaced it in modern industry?**
  1. **General Purpose:** MATLAB is a domain-specific language (DSL). You cannot easily build a web server or a game in MATLAB. Python allows you to integrate your numerical analysis directly into a production application (e.g., a Django backend).
  2. **Licensing:** MATLAB is proprietary and expensive. Python is open-source.
  3. **Namespace Management:** MATLAB defaults to a global namespace (dangerous for large software). Python uses explicit import mechanisms (import numpy as np), promoting better software engineering practices.

MATLAB defaults to a global namespace

- **In MATLAB, when you run a script:**
  - Variables and functions are placed in a shared workspace (global-like behavior).
  - Any script or function can accidentally overwrite existing variables or functions.

```
mean = 5;      % overwrites MATLAB's built-in mean() function
x = mean([1 2 3]);  % ERROR or unexpected behavior
```

- **Why this is dangerous for large software:**
  - Name conflicts are easy.
  - Debugging becomes hard.
  - Different files may unintentionally affect each other.
  - Poor modularity and maintainability.

Python uses explicit import mechanisms

- In Python, nothing is available unless you explicitly import it.
- Names are kept inside modules and namespaces.
- mean belongs to the numpy namespace (np.mean)
- It does not pollute the global namespace

❑ **Python's approach:**

- Avoids name collisions
- Improves code readability
- Encourages modular design
- Makes large projects safer and easier to maintain

```python
import numpy as np


x = np.mean([1, 2, 3])
```

Memory Architecture: The "Boxed" vs. "Unboxed" Reality

- **We must understand how data is stored in RAM.**
- **Python Lists (The Array of Pointers):**
  - A Python list, e.g., L = [1, 2, 3], is not a contiguous block of integers.
  - It is a contiguous block of pointers (memory addresses).
  - Each pointer refers to a full Python Object (PyObject) scattered elsewhere in the heap.
- **The Cost: To read L[0], the CPU must:**
  1. **Fetch** the address of the list.
  2. **Fetch** the address stored at index 0.
  3. **Jump** to that address (Pointer Chase).
  4. **Read** the PyObject (which contains reference counts, type info, and the value).
- **Result:** **Poor Cache Locality**. The CPU cache misses frequently because data is scattered.

NumPy Arrays (The Contiguous Block)

- **An ndarray allocates a single, solid block of memory (like a C array).**
- **The data is unboxed**. An int32 array stores raw 32-bit integers side-by-side **“**Stored **contiguously** in memory**”.**
- **The Benefit:** To read arr[0], the CPU reads the base address and adds the offset.
- **Result: Excellent Cache Locality**. **When arr[0] is loaded into the CPU cache, arr[1], arr[2], etc., are likely loaded with it**.

Quantitative Analysis: The Overhead

- Let us prove this mathematically. Consider storing an integer.
- In C/NumPy: A standard int64 takes exactly 8 bytes.
- **In Pure Python: An integer is an object. It contains:**
  - **ob_refcnt (Reference count):** 8 bytes
  - **ob_type (Type pointer):** 8 bytes
  - **ob_digit (The actual value):** ~8-12 bytes
  - **Total:** A single Python integer can consume 28+ bytes.
- **The List Overhead:**
  - **A list of N items requires:**
    - Space for the list structure (size, allocated slots).
    - **N x 8** bytes for the pointers.
    - **N x 28** bytes for the integer objects themselves.
- **NumPy: Requires a small header (metadata) + N x 8 bytes.**

Performance Metrics (The timeit Proof)

```python
import numpy as np
import time

size = 1_000_000  # Creates: size = 1000000

# 1. Pure Python List
list_a = list(range(size))      # Creates: [0, 1, 2, ..., 999999]
list_b = list(range(size))      # Creates: [0, 1, 2, ..., 999999]

start = time.time()
list_c = [a + b for a, b in zip(list_a, list_b)]  # Creates: [0, 2, 4, ..., 1999998]
print(f"Python List Time: {time.time() - start:.5f} sec")
# Output example: Python List Time: 0.12345 sec
```

```python
# 2. NumPy Array
arr_a = np.arange(size)         # Creates: array([0, 1, 2, ..., 999999])
arr_b = np.arange(size)         # Creates: array([0, 1, 2, ..., 999999])

start = time.time()
arr_c = arr_a + arr_b           # Creates: array([0, 2, 4, ..., 1999998])
print(f"NumPy Array Time: {time.time() - start:.5f} sec")
# Output example: NumPy Array Time: 0.00321 sec

"""
FINAL OUTPUT (example):
Python List Time: 0.12345 sec
NumPy Array Time: 0.00321 sec

NumPy is ~38x faster (0.12345 / 0.00321 ≈ 38)
"""
```

Array Creation Routines

- **np.array(object, dtype=None)**
  - Unlike lists, NumPy arrays are **homogenous**. You cannot have an integer and a string in the same numeric array without falling back to a slow object type.
  - **Slow object** means that NumPy is forced to store each array element as a **regular Python object**, which makes operations **much slower and more memory-hungry** compared to normal NumPy numeric arrays.
  - **Upcasting:** If you try **np.array([1, 2, 3.5])**, NumPy will "upcast" all integers to floats to maintain homogeneity.
- **Understanding dtype (Data Types):**
  - **Memory is finite**. As computer scientists, you must choose the right type.
  - **int8:** -128 to 127 (1 byte). Use for small masks or images.
  - **int64:** Standard 64-bit integer (8 bytes).

Interval Generation

- **np.arange([start , stop, step, dtype=None)**
  - Similar to Python's **range()**, but returns an array.
  - **The Danger:** Be very careful using floating-point steps.
  - **Example:** np.arange(0, 0.3, 0.1) might create [0., 0.1, 0.2] or [0., 0.1, 0.2, 0.3] depending on machine precision error accumulation.
    - **Result length becomes unpredictable → dangerous in scientific code**.

Interval Generation (Cont.)

- **np.linspace(start, stop, num=50, endpoint=True, retstep=False)**
  - **The safe solution for floats**
  - Instead of saying “step size”, you say: “Give me exactly N evenly spaced points”.
    - Exact number of points
    - No accumulated rounding error
    - Predictable output length

```
np.linspace(0, 0.3, num=3)
# array([0. , 0.15, 0.3])
```

  - If **retstep=True**, It Returns the actual step size

Interval Generation (Cont.)

- **endpoint=True** (default)Includes stop value.

```
np.linspace(0, 1, 5)
# [0. , 0.25, 0.5 , 0.75, 1. ]
```

- **endpoint=False,** Excludes the last value.
  - **Common in signal processing / periodic waves.**
  - **Prevents duplicating the first point of the next cycle.**

```
result = np.linspace(0, 2*np.pi, 4, endpoint=False)
print(result)


# Output: array([0.        , 1.57079633, 3.14159265, 4.71238898])
```

Initialization Strategies

- **np.zeros(shape) / np.ones(shape):**
  - Allocates memory and fills it with 0s or 1s.

    Safe default.
- **Why they are “safe”**
  - **No undefined values**
  - **Predictable results**
  - **Ideal for teaching, debugging, and general use.**
  - **Slightly slower because NumPy must write values to memory.**

```
np.zeros((2, 3))
# [[0. 0. 0.]
#  [0. 0. 0.]]

np.ones((2, 3))
# [[1. 1. 1.]
#  [1. 1. 1.]]
```

Initialization Strategies (Cont.)

- **np.empty(shape):**
  - **Warning:** This allocates memory without initializing it.
  - It effectively returns whatever "garbage" values happened to be at that memory address.
  - **Use Case:** High-performance scenarios **where you guarantee you will overwrite every single value immediately**, and you don't want to waste CPU cycles writing zeros first.

```
np.empty((2, 3))
# [[ 6.9e-310  2.1e-314  9.9e-324]
#  [ 1.3e-313  5.6e-311  2.0e-314]]
```

Initialization Strategies (Cont.)

- **When np.empty() is appropriate (use case)**
  - **High-performance code ONLY**
- **Use it only if:**
  - You will overwrite every element
  - Immediately after allocation
  - No element will ever be read uninitialized
- **Why this is dangerous**
  - Values are unpredictable
  - They change between runs and machines
  - Using them before overwriting causes bugs that are hard to detect
- **Never assume np.empty() gives zeros.**

Initialization Strategies (Cont.)

- **np.identity(n):**
  - Returns a square **n x n** matrix with **1s** on the main diagonal.

```
I3 = np.identity(3)
print("np.identity(3):")
print(I3)
# Output: [[1. 0. 0.]
#          [0. 1. 0.]
#          [0. 0. 1.]]

print("\nnp.identity(4):")
print(np.identity(4))
# Output: [[1. 0. 0. 0.]
#          [0. 1. 0. 0.]
#          [0. 0. 1. 0.]
#          [0. 0. 0. 1.]]
```

Initialization Strategies (Cont.)

- **np.eye(N, M=None, k=0):**
  - **More flexible**. Can create rectangular matrices.
  - **Parameter k:** Controls the diagonal offset.
    - **k=0:** Main diagonal.
    - **k=1:** Upper diagonal (Superdiagonal).
    - **k=-1:** Lower diagonal (Subdiagonal).
- **Application:** Useful for linear algebra operations, such as creating tridiagonal matrices for solving differential equations.

Initialization Strategies (Cont.)

```
print(np.eye(3))
# Output: [[1. 0. 0.]
#          [0. 1. 0.]
#          [0. 0. 1.]]

print("\n2. Rectangular matrix:")
print("np.eye(2, 4):")  # 2 rows, 4 columns
print(np.eye(2, 4))
# Output: [[1. 0. 0. 0.]
#          [0. 1. 0. 0.]]

print("\nnp.eye(4, 2):")  # 4 rows, 2 columns
print(np.eye(4, 2))
# Output: [[1. 0.]
#          [0. 1.]
#          [0. 0.]
#          [0. 0.]]
```

Initialization Strategies (Cont.)

```
matrix = np.eye(5, 5)  # 5x5 identity
print("Original 5x5 identity:")
print(matrix)
# Output: [[1. 0. 0. 0. 0.]
#          [0. 1. 0. 0. 0.]
#          [0. 0. 1. 0. 0.]
#          [0. 0. 0. 1. 0.]
#          [0. 0. 0. 0. 1.]]

print("\n1. Main diagonal (k=0, default):")
print("np.eye(4, 4, k=0):")
print(np.eye(4, 4, k=0))
# Output: [[1. 0. 0. 0.]
#          [0. 1. 0. 0.]
#          [0. 0. 1. 0.]
#          [0. 0. 0. 1.]]
```

Initialization Strategies (Cont.)

```
print("\n2. Upper diagonal (k=1):")
print("np.eye(4, 4, k=1):")
print(np.eye(4, 4, k=1))
# Output: [[0. 1. 0. 0.]
#          [0. 0. 1. 0.]
#          [0. 0. 0. 1.]
#          [0. 0. 0. 0.]]

print("\n3. Lower diagonal (k=-1):")
print("np.eye(4, 4, k=-1):")
print(np.eye(4, 4, k=-1))
# Output: [[0. 0. 0. 0.]
#          [1. 0. 0. 0.]
#          [0. 1. 0. 0.]
#          [0. 0. 1. 0.]]
```

```
print("\n4. Further offsets:")
print("np.eye(5, 5, k=2):")
print(np.eye(5, 5, k=2))
# Output: [[0. 0. 1. 0. 0.]
#          [0. 0. 0. 1. 0.]
#          [0. 0. 0. 0. 1.]
#          [0. 0. 0. 0. 0.]
#          [0. 0. 0. 0. 0.]]

print("\nnp.eye(5, 5, k=-2):")
print(np.eye(5, 5, k=-2))
# Output: [[0. 0. 0. 0. 0.]
#          [0. 0. 0. 0. 0.]
#          [1. 0. 0. 0. 0.]
#          [0. 1. 0. 0. 0.]
#          [0. 0. 1. 0. 0.]]
```

Array Manipulation, Indexing, and Memory Layout

Attributes of the N-Dimensional Array

- An ndarray is defined by its metadata. The data buffer is just a bluk of bytes; the metadata tells the CPU how to interpret it.
  - **ndim (Number of Dimensions):** The rank of the array.

ndim	Meaning	Example
0	Scalar	np.array(5)
1	Vector	[1, 2, 3]
2	Matrix	[[1,2],[3,4]]
3	Tensor	3D data (e.g., images, volumes)

```
a = np.array([[1, 2], [3, 4]])
a.ndim   # 2
```

  - **Metadata defines:**
    - **ndim:** number of dimensions
    - **shape:** size along each axis

Attributes of the N-Dimensional Array

- **shape (Tuple of Integers):** The exact size along each axis. For a matrix with R rows and C columns, the shape is (R, C).

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
a.shape
# (2, 3)
```

Attributes of the N-Dimensional Array

- **When you change the shape of a NumPy array, only the metadata is modified, not the actual data, as long as the total number of elements stays the same.**
- **What actually happens**
  - NumPy does not move or copy data
  - It only updates the shape metadata
  - The same data buffer is re-interpreted with a new layout
- **Important Concepts:**
  - Changing shape ≠ changing data
  - Same bytes, different interpretation
  - Extremely fast (metadata-only change)
  - **Fails if element count mismatches**

```
a = np.arange(6)
# [0 1 2 3 4 5]

a.shape = (2, 3)

[[0 1 2]
 [3 4 5]]
```

Reshaping: The Art of Interpretation

- **arr.reshape(new_shape):**
  - This is one of the most powerful commands in NumPy. It changes the dimensionality without moving any data in memory.
  - **Constraint: The total number of elements must remain constant. You cannot reshape a size-10 array into (3, 4) (size 12).**
- **Using reshape() (recommended):**
  - Returns a view when possible
  - Safer than directly modifying shape
  - **reshape() returns a new array object that shares the same data (a view), while keeping the original array unchanged**

Reshaping: The Art of Interpretation (Cont.)

```
print("Original 1D array:")
print(a)
# Output: [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10. 11.]


print()


# Reshape to 2D (creates a view)
b = a.reshape(3, 4)
print("Reshaped to 3x4 (VIEW):")
print(b)
# Output:
# [[ 0.  1.  2.  3.]
#  [ 4.  5.  6.  7.]
#  [ 8.  9. 10. 11.]]
```

Reshaping: The Art of Interpretation

❑ Key Differences Table

Aspect	shape attribute	reshape() method
Type	Attribute (property)	Method (function)
Action	Modifies array in-place	Returns new view of data
Original	Original array is changed	Original array unchanged
Syntax	arr.shape = (3, 4)	new_arr = arr.reshape(3, 4)
Memory	Same memory, reshaped	Same memory (view), new shape
Safety	Risky (changes original)	Safer (preserves original)

Flattening in NumPy: flatten() vs. ravel()

- **arr.flatten()**
  - Always returns a deep copy
  - Allocates new memory and copies all data
  - Safe but slower because it duplicates the array
- **arr.ravel()**
  - Returns a view when possible
  - No new memory allocated, just presents existing data as 1D
  - Fast and memory-efficient
  - **Warning: Changes to the raveled array affect the original (if it's a view)**
- **Rule of thumb: Use ravel() for performance when you won't modify or when modifications to the original are acceptable. Use flatten() when you need a completely independent 1D copy.**

Flattening in NumPy: flatten() vs. ravel()

arr.ravel() vs arr.reshape(n)

1. arr.reshape(-1) or arr.reshape(n)

When you use reshape, you are telling NumPy: "I want this specific shape."

- If you have a 3 X 3 array (9 elements), reshape(9) or reshape(-1) will give you a 1D array of length 9.
- **The Difference:** You have to know the size (or use -1 to let NumPy calculate it).

2. arr.ravel()

This is a "convenience" function. It doesn't care how many elements are in the array; it simply unrolls everything into a 1D view.

- **The Similarity:** Like reshape(), ravel() also returns a view whenever possible, meaning if you change the raveled array, the original changes too!

Flattening in NumPy: flatten() vs. ravel() (Cont.)

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Original 2D array:")
print(arr)
# Output:
# [[1 2 3]
#  [4 5 6]]

print(f"\nOriginal array ID: {id(arr)}")
# Output: Original array ID: 139716123456789

# ================= flatten() =================
flattened = arr.flatten()
print("\n=== Using flatten() ===")
print("Flattened array:", flattened)
# Output: Flattened array: [1 2 3 4 5 6]
```

Flattening in NumPy: flatten() vs. ravel() (Cont.)

```
raveled = arr.ravel()
print("\n=== Using ravel() ===")
print("Raveled array:", raveled)
# Output: Raveled array: [1 2 3 4 5 6]

print(f"Is view? {raveled.base is arr}")  # True = view of arr
# Output: Is view? True
print(f"Same memory? {raveled.base is arr}")
# Output: Same memory? True

# Modify raveled - original IS affected (because it's a view)
raveled[0] = 100
print(f"\nAfter raveled[0] = 100:")
print(f"Raveled: {raveled}")
# Output: Raveled: [100   2   3   4   5   6]
print(f"Original:\n{arr}")
# Output:
# Original:
# [[100   2   3]
#  [  4   5   6]]
```

Flattening in NumPy: flatten() vs. ravel() (Cont.)

Aspect	flatten()	ravel()
Returns	Always a copy	View when possible
Memory	New allocation	Shares memory (if possible)
Speed	Slower (copies data)	Faster (no copy usually)
Safety	Safe (no side effects)	Risky (may affect original)
Use when	Need independent array	Need memory efficiency

Memory Layout: C-style vs. Fortran-style

- **Memory is linear (1D). How do we map a 2D matrix to 1D RAM?**
- **Row-Major ('C' Order - Default):**
  - We traverse the last axis first. We move across the row, then jump to the next row.
  - [[1, 2], [3, 4]] → stored as 1, 2, 3, 4.
- **Column-Major ('F' Order - Fortran/MATLAB):**
  - We traverse the first axis first. We move down the column, then jump to the next column.
  - [[1, 2], [3, 4]] → stored as 1, 3, 2, 4.
- **Performance Implication:** Algorithms are faster when they traverse memory sequentially. If you iterate an array by columns, **you trigger cache misses, slowing down your code significantly**.

Memory Layout: C-style vs. Fortran-style

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("3x3 Matrix:")
print(matrix)
# Output:
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]

print("\n=== Memory Layout ===")

# C-order (row-major) - DEFAULT
print("C-order (row-major) layout:")
c_array = np.array(matrix, order='C')
print(f"Memory order: {c_array.flatten()}")
# Output: Memory order: [1 2 3 4 5 6 7 8 9]

# F-order (column-major)
print("\nF-order (column-major) layout:")
f_array = np.array(matrix, order='F')
print(f"Memory order: {f_array.flatten()}")
# Output: Memory order: [1 4 7 2 5 8 3 6 9]
```

Slicing and Views

Slicing Semantics

- **Syntax:** array[start : stop : step]
- In multi-dimensional arrays, we slice each axis separated by commas: matrix[0:2, 1:5].
- Ellipsis (...): A shorthand to mean "all other dimensions." arr[..., 0] takes the 0th element of the very last axis, regardless of how many dimensions arr has.

Example 1 - Basic Slicing

```python
import numpy as np

# Create a 1D array
arr_1d = np.arange(10)
print("1D array:", arr_1d)
# Output: 1D array: [0 1 2 3 4 5 6 7 8 9]

print("\n=== Basic 1D Slicing ===")
print("arr_1d[2:6]:", arr_1d[2:6])        # Elements 2 to 5
# Output: arr_1d[2:6]: [2 3 4 5]

print("arr_1d[::2]:", arr_1d[::2])        # Every other element
# Output: arr_1d[::2]: [0 2 4 6 8]

print("arr_1d[::-1]:", arr_1d[::-1])      # Reverse
# Output: arr_1d[::-1]: [9 8 7 6 5 4 3 2 1 0]

print("arr_1d[-3:]:", arr_1d[-3:])        # Last 3 elements
# Output: arr_1d[-3:]: [7 8 9]
```

Example 2 - Multi-dimensional Slicing

```
# Create a 2D array (3x4)
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
print("\n2D Matrix (3x4):")
print(matrix)
# Output:
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

print("\n=== 2D Slicing ===")
print("matrix[0:2, 1:5]:")  # Rows 0-1, Columns 1-3
print(matrix[0:2, 1:4])
# Output:
# [[2 3 4]
#  [6 7 8]]
```

```
print("\nmatrix[:, 2]:")    # All rows, column 2
print(matrix[:, 2])
# Output: matrix[:, 2]: [ 3  7 11]

print("\nmatrix[1, :]:")    # Row 1, all columns
print(matrix[1, :])
# Output: matrix[1, :]: [5 6 7 8]

print("\nmatrix[::2, ::2]:")  # Every other row and column
print(matrix[::2, ::2])
# Output:
# [[ 1  3]
#  [ 9 11]]
```

Example 3 - Ellipsis (...) Power

```python
import numpy as np

# Create a simple RGB image: 3x3 pixels with 3 color channels (R, G, B)
# Shape: (height=3, width=3, channels=3)
image = np.array([
    [[255, 0, 0],    [0, 255, 0],    [0, 0, 255]],   # Row 0: Red, Green, Blue
    [[255, 255, 0],  [255, 0, 255],  [0, 255, 255]], # Row 1: Yellow, Magenta, Cyan
    [[128, 128, 128],[64, 64, 64],   [0, 0, 0]]      # Row 2: Grays
])

print("Image shape:", image.shape)
print("Image array:")
print(image)
# Output:
# [[[255   0   0]
#   [  0 255   0]
#   [  0   0 255]]
#  [[255 255   0]
#   [255   0 255]
#   [  0 255 255]]
#  [[128 128 128]
#   [ 64  64  64]
#   [  0   0   0]]]

print("\n=== WITHOUT Ellipsis ===")
red_channel = image[:, :, 0]      # All rows, all columns, channel 0 (Red)
print("Red channel (image[:, :, 0]):")
print(red_channel)
# Output:
# [[255   0   0]
#  [255 255   0]
#  [128  64   0]]

print("\n=== WITH Ellipsis ===")
red_channel_easy = image[..., 0]  # Same as above but cleaner!
print("Red channel (image[..., 0]):")
print(red_channel_easy)
# Output: Same as above!

print("\n=== Extract all Green values ===")
green_values = image[..., 1]      # Channel 1 = Green
print("Green values (image[..., 1]):")
print(green_values)
# Output:
# [[  0 255   0]
#  [255   0 255]
#  [128  64   0]]
```

Why NumPy Slices Are Views (Not Copies)

- Unlike Python lists, where **new_list = old_list[:]** creates a copy, **NumPy slices are views.**
- **Why? NumPy assumes you are working with gigabytes of data. Copying a 10GB array just to look at the first row is inefficient**.

```
import numpy as np
A = np.array([1, 2, 3, 4])
B = A[0:2]  # B is a VIEW of A
B[0] = 99
print(A)    # Output: [99, 2, 3, 4] -> A was modified!
```

Why NumPy Uses Views Instead of Copies

- **Performance with Massive Data.**
  - A view avoids copying completely. It simply creates a new object that points to the same memory, with different indexing metadata (called strides).
- **Strides Make This Possible.**
  - A NumPy array knows how to walk over memory using a concept called strides.
  - A slice just creates a different set of strides, not different data.
- **NumPy Follows the "Do Nothing Expensive Automatically" Philosophy.**
  - If you want a copy, you should ask for one. **b = a[::2].copy()**

This avoids accidental performance penalties in scientific and large-scale data applications.

Why NumPy Uses Views Instead of Copies (Cont.)

Strides

Strides is a tuple of integers representing the **number of bytes** you must "step" or "jump" in memory to move to the next element along each axis.

How Strides Work (A 2D Example)

- Imagine a 2 X 3 array of 64-bit integers (each number takes 8 bytes).
- **In memory**, it looks like this **flat line: [10, 20, 30, 40, 50, 60]**
- If this is in **C-order** (row-major), the strides will be **(24, 8)**:
  - **8 (for Axis 1/Columns):** To move from 10 to 20, you just jump 8 bytes (1 element) forward.
  - **24 (for Axis 0/Rows):** To move from 10 to 40 (the next row), you have to jump over three numbers (3 X 8 bytes = 24 bytes).

```
matrix = np.array([[10, 20, 30],
                   [40, 50, 60]])
```

Advanced Indexing - Boolean Indexing (Masking)

- **NumPy Array Filtering Using Boolean Conditions**
- **Concept**
  - Boolean indexing allows selecting elements based on **conditions**, **not positions**.
  - Instead of integers, we provide an array of True/False values called a mask.
  - Only elements where the mask is True are returned.
- **Key Properties**
  - Output is always 1D, regardless of original shape.
  - Mask must match the shape of the array being indexed.
  - Supports compound conditions using **&**, **|**, **~**.
- **Use Cases**
  - Data cleaning
  - Extracting subsets efficiently
  - Conditional filtering in scientific computing

```
data = np.array([10, 20, 30, 40])
mask = data > 25  # [False, False, True, True]
filtered = data[mask] # [30, 40]
```

Example 1 - Boolean Indexing

```python
import numpy as np

# Create a simple array
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print("Original array:", arr)
# Output: Original array: [1 2 3 4 5 6 7 8 9]

# Create a boolean mask
mask = np.array([True, False, True, False, True, False, True, False, True])
print("Boolean mask:", mask)
# Output: Boolean mask: [ True False  True False  True False  True False  True]

# Apply the mask
result = arr[mask]
print("Elements where mask is True:", result)
# Output: Elements where mask is True: [1 3 5 7 9]
```

Example 2 - Creating Masks with Conditions

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Create masks using comparison operators
even_mask = arr % 2 == 0
greater_mask = arr > 5
multiple_of_3_mask = arr % 3 == 0

print("Even numbers mask (arr % 2 == 0):", even_mask)
print("Even numbers:", arr[even_mask])
# Output: Even numbers mask: [False  True False  True False  True False  True False  True]
# Output: Even numbers: [ 2  4  6  8 10]

print("\nNumbers > 5 mask (arr > 5):", greater_mask)
print("Numbers > 5:", arr[greater_mask])
# Output: Numbers > 5 mask: [False False False False False  True  True  True  True  True]
# Output: Numbers > 5: [ 6  7  8  9 10]

print("\nMultiple of 3 mask (arr % 3 == 0):", multiple_of_3_mask)
print("Multiple of 3:", arr[multiple_of_3_mask])
# Output: Multiple of 3 mask: [False False  True False False  True False False  True False]
# Output: Multiple of 3: [3 6 9]
```

Example 2 - Creating Masks with Conditions (Cont.)

```
# Combine masks with logical operators
print("\n=== Combining Masks ===")
combined_mask = (arr > 3) & (arr < 8)  # AND: numbers between 3 and 8
print("(arr > 3) & (arr < 8):", combined_mask)
print("Numbers between 3 and 8:", arr[combined_mask])
# Output: (arr > 3) & (arr < 8): [False False False  True  True  True  True False False False]
# Output: Numbers between 3 and 8: [4 5 6 7]

or_mask = (arr < 3) | (arr > 8)  # OR: numbers less than 3 OR greater than 8
print("\n(arr < 3) | (arr > 8):", or_mask)
print("Numbers < 3 OR > 8:", arr[or_mask])
# Output: (arr < 3) | (arr > 8): [ True  True False False False False False False  True  True]
# Output: Numbers < 3 OR > 8: [ 1  2  9 10]

not_mask = ~even_mask  # NOT: not even (odd numbers)
print("\nNOT even mask (~even_mask):", not_mask)
print("Odd numbers:", arr[not_mask])
# Output: NOT even mask: [ True False  True False  True False  True False  True False]
# Output: Odd numbers: [1 3 5 7 9]
```

Example 3 - 2D Array Masking

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
print("2D Matrix:")
print(matrix)
# Output:
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]

# Create a 2D mask
mask_2d = matrix > 4
print("\nMask (matrix > 4):")
print(mask_2d)
# Output:
# [[False False False]
#  [False  True  True]
#  [ True  True  True]]

# Apply mask - returns 1D array!
result = matrix[mask_2d]
print("\nElements where mask is True:", result)
# Output: Elements where mask is True: [5 6 7 8 9]

# Mask rows or columns
print("\n=== Masking Rows and Columns ===")
row_mask = np.array([True, False, True])  # Keep rows 0 and 2
col_mask = np.array([False, True, True])  # Keep columns 1 and 2
```

Example 3 - 2D Array Masking (Cont.)

```
# Mask rows or columns
print("\n=== Masking Rows and Columns ===")
row_mask = np.array([True, False, True])  # Keep rows 0 and 2
col_mask = np.array([False, True, True])  # Keep columns 1 and 2

print("Row mask:", row_mask)
print("Column mask:", col_mask)

# Apply row mask
rows_selected = matrix[row_mask, :]
print("\nRows selected (matrix[row_mask, :]):")
print(rows_selected)
# Output:
# [[1 2 3]
#  [7 8 9]]

# Apply column mask
cols_selected = matrix[:, col_mask]
print("\nColumns selected (matrix[:, col_mask]):")
print(cols_selected)
# Output:
# [[2 3]
#  [5 6]
#  [8 9]]
```

Example 4 - Practical Examples

```
# Example 2: Image thresholding
print("\n=== Image Thresholding ===")
# Simulate grayscale image (4x4 pixels, values 0-255)
image = np.array([
    [100, 150, 50, 200],
    [30, 180, 220, 70],
    [160, 40, 90, 240],
    [120, 80, 140, 10]
])
print("Original image:")
print(image)

# Create binary mask for "bright" pixels (value > 128)
bright_mask = image > 128
print("\nBright pixels mask (value > 128):")
print(bright_mask)
# Output:
# [[False  True False  True]
#  [False  True  True False]
#  [ True False False  True]
#  [False False  True False]]
```

```
# Apply mask to get bright pixel values
bright_pixels = image[bright_mask]
print("\nBright pixel values:", bright_pixels)
# Output: Bright pixel values: [150 200 180 220 160 240 140]

# Set bright pixels to white (255) and dark pixels to black (0)
binary_image = np.zeros_like(image)
binary_image[bright_mask] = 255
print("\nBinary image (black & white):")
print(binary_image)
# Output:
# [[  0 255   0 255]
#  [  0 255 255   0]
#  [255   0   0 255]
#  [  0   0 255   0]]
```

Creates a new array with the same shape and data type as image, but filled with zeros.

Fancy Indexing in NumPy

- **Selecting Elements Using Integer Arrays.**
- **Definition**
  - Fancy indexing uses arrays of integer indices to extract specific elements from another array.
  - Unlike slicing, which may return a view, **fancy indexing always returns a copy of the data.**

```python
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

selected = arr[[0, 2, 4]]
print(selected)   # → [10 30 50]
```

- **Key Properties**
  - Accepts lists or NumPy arrays of integers.
  - The selected elements are gathered and placed into a **new independent array.**
  - Changes to the result do **NOT** affect the original array.

Fancy Indexing in NumPy (Cont.)

- **Why Fancy Indexing Returns a Copy?**
  - **Fancy indexing always returns a copy** because NumPy cannot guarantee that the selected elements will be **contiguous in memory** or maintain a **regular striding pattern** that would allow for a view.
- **Use Cases**
  - Random sampling
  - Extracting arbitrary elements
  - Selecting rows/columns by index arrays
  - Advanced manipulation in machine learning & data processing

Example 1 – Basic Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90])
print("Original array:", arr)
# Output: Original array: [10 20 30 40 50 60 70 80 90]

print("\n=== Basic Fancy Indexing ===")

# Pick specific elements by their indices
result = arr[[0, 2, 4]]  # Indices 0, 2, 4
print("arr[[0, 2, 4]]:", result)
# Output: arr[[0, 2, 4]]: [10 30 50]

# Indices can be in any order
result = arr[[4, 0, 2, 7]]  # Indices 4, 0, 2, 7
print("arr[[4, 0, 2, 7]]:", result)
# Output: arr[[4, 0, 2, 7]]: [50 10 30 80]
```

```
# Indices can repeat
result = arr[[0, 0, 1, 1, 2, 2]]  # Repeated indices
print("arr[[0, 0, 1, 1, 2, 2]]:", result)
# Output: arr[[0, 0, 1, 1, 2, 2]]: [10 10 20 20 30 30]

# Negative indices work too
result = arr[[-1, -2, -3]]  # Last, second-last, third-last
print("arr[[-1, -2, -3]]:", result)
# Output: arr[[-1, -2, -3]]: [90 80 70]
```

Example 2 – 2D Array Fancy Indexing

```
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])
print("Matrix (3x4):")
print(matrix)
# Output:
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

print("\n1. Selecting rows with fancy indexing:")
# Select rows 2, 0, 1 (in that order)
rows = matrix[[2, 0, 1]]
print("matrix[[2, 0, 1]] (rows 2, 0, 1):")
print(rows)
# Output:
# [[ 9 10 11 12]
#  [ 1  2  3  4]
#  [ 5  6  7  8]]

print("\n2. Selecting specific elements from each row:")
# First element from row 0, second from row 1, third from row 2
result = matrix[[0, 1, 2], [0, 1, 2]]  # (row indices, column indices)
print("matrix[[0, 1, 2], [0, 1, 2]]:")
print("Gets elements: (0,0), (1,1), (2,2)")
print("Result:", result)
# Output: Result: [1 6 11]
```

Example 2 – 2D Array Fancy Indexing (Cont.)

```
print("\n3. Selecting columns with fancy indexing:")
# Select columns 3, 1, 0 (in that order)
columns = matrix[:, [3, 1, 0]]
print("matrix[:, [3, 1, 0]] (columns 3, 1, 0):")
print(columns)
# Output:
# [[ 4  2  1]
#  [ 8  6  5]
#  [12 10  9]]

print("\n4. Creating a submatrix with fancy indexing:")
# Select rows 0 and 2, columns 1 and 3
submatrix = matrix[[0, 2]][:, [1, 3]]  # First select rows, then columns
print("matrix[[0, 2]][:, [1, 3]]:")
print(submatrix)
# Output:
# [[ 2  4]
#  [10 12]]

# More concise way:
submatrix = matrix[np.ix_([0, 2], [1, 3])]
print("\nUsing np.ix_ for cleaner indexing:")
print("matrix[np.ix_([0, 2], [1, 3])]:")
print(submatrix)
# Output: Same as above
```

Example 3 – Multi-dimensional Fancy Indexing

```
# Create a 3D array (2x3x4)
arr_3d = np.arange(24).reshape(2, 3, 4)
print("3D array shape:", arr_3d.shape)
print("Array content:")
print(arr_3d)
# Output:
# [[[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]
#  [[12 13 14 15]
#   [16 17 18 19]
#   [20 21 22 23]]]

print("\n1. Fancy indexing first dimension:")
result = arr_3d[[1, 0]]  # Swap the two "blocks"
print("arr_3d[[1, 0]] (swap blocks):")
print(result)
# Output:
# [[[12 13 14 15]
#   [16 17 18 19]
#   [20 21 22 23]]
#  [[ 0  1  2  3]
#   [ 4  5  6  7]
#   [ 8  9 10 11]]]
```

```
print("\n2. Fancy indexing multiple dimensions:")
# For each "block", pick specific rows and columns
result = arr_3d[[0, 1], [0, 2], [1, 3]]
print("arr_3d[[0, 1], [0, 2], [1, 3]]:")
print("Gets: block0,row0,col1 =", arr_3d[0, 0, 1])
print("      block1,row2,col3 =", arr_3d[1, 2, 3])
print("Result:", result)
# Output: Result: [ 1 23]
```

Fancy Indexing vs Slicing — Comparison Table

Feature	Slicing	Fancy Indexing
Definition	Selects **continuous blocks** of data using start:stop:step	Selects **arbitrary elements** using lists/arrays of integers
Memory Behavior	**Returns a view** (shares memory with original array)	**Returns a copy** (independent array)
Performance	**Very fast** (no data copying)	**Slower** (must allocate new array and gather elements)
Memory Usage	**Minimal** (no duplication)	**Higher** (creates a new array)
Index Type	Slices (: syntax)	Integer arrays or lists
Example Syntax	arr[1:5]	arr[[0, 3, 7]]
Effect on Original Array	Modifying slice **modifies original**	Modifying result does **NOT modify original**
Supported Dimensions	Works naturally across dimensions	Supports advanced patterns like row/column selection via index arrays
Use Cases	**Continuous subarrays** (e.g., rows, ranges)	**Arbitrary element selection**, sampling, reordering

Non-Zero Element Retrieval in NumPy

- **np.nonzero(arr)**
- **Definition**
  - Returns a tuple of arrays, one per dimension.
  - Each array contains the indices of non-zero elements along that dimension.
- **Key Characteristics**
  - Output length = number of dimensions (ndim)
  - Indices are aligned element-wise across dimensions
  - Preserves the original array shape logic

Example 1 – Basic Non-Zero Element Retrieval

```
arr = np.array([0, 5, 0, 3, 0])
np.nonzero(arr)
```

```
(array([1, 3]),)
```

Example 2 – 2D array

```
arr_2d = np.array([[0, 1, 0],
                   [2, 0, 3],
                   [0, 0, 4]])
print("2D array:")
print(arr_2d)
# Output:
# [[0 1 0]
#  [2 0 3]
#  [0 0 4]]

nonzero_2d = np.nonzero(arr_2d)
print("\nnp.nonzero(arr_2d):", nonzero_2d)
print("Row indices:", nonzero_2d[0])
print("Column indices:", nonzero_2d[1])

# Access the non-zero elements
print("Non-zero elements:", arr_2d[nonzero_2d])
# Output: np.nonzero(arr_2d): (array([0, 1, 1, 2]), array([1, 0, 2, 2]))
# Output: Row indices: [0 1 1 2]
# Output: Column indices: [1 0 2 2]
# Output: Non-zero elements: [1 2 3 4]
```

Non-Zero Element Retrieval in NumPy

- **np.flatnonzero(arr)**
- **Definition**
  - Flattens the array into 1D
  - Returns indices of non-zero elements in the flattened array
- **Key Characteristics**
  - Always returns a 1D array
  - Ignores original dimensional structure
  - Equivalent to **np.nonzero(arr.ravel())[0]**
    - **arr.ravel():** Converts arr into a **1D array.**
    - **np.nonzero(arr.ravel()):** Finds indices of non-zero elements. Since the array is 1D, the result is a tuple with one element.
    - **[0]:** Extracts the only array from the tuple.

Example 3 – Flattened Version

```
arr_2d = np.array([[0, 1, 0],
                   [2, 0, 3],
                   [0, 0, 4]])
print("2D array:")
print(arr_2d)

# flatnonzero returns indices in flattened (1D) array
flat_indices = np.flatnonzero(arr_2d)
print("\nnp.flatnonzero(arr_2d):", flat_indices)
print("Flattened array:", arr_2d.ravel())
print("Indices in flattened array where elements are non-zero")

# Verify by checking flattened array
flattened = arr_2d.ravel()
print("\nVerification:")
for idx in flat_indices:
    print(f"  flattened[{idx}] = {flattened[idx]}")
# Output: np.flatnonzero(arr_2d): [1 3 5 8]
# Output: Flattened array: [0 1 0 2 0 3 0 0 4]
# Output:
#   flattened[1] = 1
#   flattened[3] = 2
#   flattened[5] = 3
#   flattened[8] = 4
```

Numerical Operations and Linear Algebra

Vectorization and Arithmetic

- **Element-wise Operations**
  - NumPy overrides standard operators (+, -, *, /).
  - **The Concept:** When you write A + B, NumPy does not iterate in Python. It delegates to a C-loop that uses CPU vector instructions (AVX/SSE) to add chunks of data simultaneously.

```
A = np.array([1, 2, 3])
B = np.array([4, 5, 6])

A + B
# array([5, 7, 9])
```

Vectorization and Arithmetic

- **Performance Concept (Important)**
  - When you write A + B, NumPy does not loop in Python.
  - **What Actually Happens**
    - Uses vectorized CPU instructions (SIMD)
    - Processes multiple elements per CPU cycle
- **Result**
  - Faster execution
  - No Python-level loops
  - Efficient cache usage

This is the core idea behind NumPy vectorization

Vectorization and Arithmetic

❑ Matrices vs. 2D Arrays in NumPy

- ➢ np.array (Standard & Recommended)
- ➢ General-purpose N-dimensional array
- ➢ Most commonly used NumPy data structure

❑ Multiplication Behavior

- ➢ Element-wise multiplication **(Hadamard Product)**

```
A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

A * B
# [[ 5 12]
#  [21 32]]
```

Vectorization and Arithmetic

- **Recommended Practice**
  - Avoid **np.matrix.** It is legacy, confusing, and discouraged.
- **Use np.array for everything**
  - Clear semantics
  - Consistent behavior
  - Compatible with all scientific libraries
- **Use @ for Matrix Multiplication (Introduced in Python 3.5)**
  - (A @ B) or np.matmul(A, B)

Example 1 - Element-wise Operations

```
A = np.array([1, 2, 3, 4])
B = np.array([5, 6, 7, 8])
print("Array A:", A)
print("Array B:", B)
# Output: Array A: [1 2 3 4]
# Output: Array B: [5 6 7 8]

print("\n=== Basic Element-wise Operations ===")

# Addition
print("A + B =", A + B)
# Output: A + B = [ 6  8 10 12]

# Subtraction
print("A - B =", A - B)
# Output: A - B = [-4 -4 -4 -4]

# Multiplication
print("A * B =", A * B)
# Output: A * B = [ 5 12 21 32]
```

```
# Division
print("B / A =", B / A)
# Output: B / A = [5.         3.         2.33333333 2.        ]

# Power
print("A ** 2 =", A ** 2)
# Output: A ** 2 = [ 1  4  9 16]

print("\n=== Element-wise with Scalars ===")
print("A + 10 =", A + 10)
print("A * 2 =", A * 2)
print("2 ** A =", 2 ** A)
# Output: A + 10 = [11 12 13 14]
# Output: A * 2 = [2 4 6 8]
# Output: 2 ** A = [ 2  4  8 16]
```

Example 2 - Why Element-wise is Fast: Vectorization

```
size = 10_000_000  # 10 million elements
arr1 = np.random.rand(size)
arr2 = np.random.rand(size)

print(f"Array size: {size:,} elements")
# Output: Array size: 10,000,000 elements

# Method 1: NumPy vectorized (fast)
start = time.time()
result_np = arr1 + arr2
time_np = time.time() - start

# Method 2: Python loop (slow)
start = time.time()
result_loop = np.empty(size)
for i in range(size):
    result_loop[i] = arr1[i] + arr2[i]
time_loop = time.time() - start

print(f"NumPy vectorized: {time_np:.6f} seconds")
# Output: NumPy vectorized: 0.020123 seconds (example - actual time will vary)

print(f"Python loop:      {time_loop:.6f} seconds")
# Output: Python loop:      1.456789 seconds (example - actual time will vary)

print(f"NumPy is {time_loop/time_np:.0f}x faster!")
# Output: NumPy is 72x faster! (example - actual ratio will vary)
```

Example 3 - Universal Functions (ufuncs)

```
arr = np.array([1, 2, 3, 4, 5])

print("Trigonometric functions:")
print("np.sin(arr) =", np.sin(arr))
print("np.cos(arr) =", np.cos(arr))
# Output: np.sin(arr) = [ 0.84147098  0.90929743  0.14112001 -0.7568025  -0.95892427]
# Output: np.cos(arr) = [ 0.54030231 -0.41614684 -0.9899925  -0.65364362  0.28366219]

print("\nExponential and logarithmic:")
print("np.exp(arr) =", np.exp(arr))
print("np.log(arr) =", np.log(arr))
# Output: np.exp(arr) = [  2.71828183   7.3890561   20.08553692  54.59815003 148.4131591 ]
# Output: np.log(arr) = [0.         0.69314718 1.09861229 1.38629436 1.60943791]

print("\nRounding:")
print("np.round([1.23, 4.56, 7.89]) =", np.round([1.23, 4.56, 7.89]))
# Output: np.round([1.23, 4.56, 7.89]) = [1. 5. 8.]

print("\nStatistical:")
print("np.mean(arr) =", np.mean(arr))
print("np.std(arr) =", np.std(arr))
print("np.sum(arr) =", np.sum(arr))
# Output: np.mean(arr) = 3.0
# Output: np.std(arr) = 1.4142135623730951
# Output: np.sum(arr) = 15
```

Example 4 - The @ Operator (Matrix Multiplication)

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])  # Shape: (2, 3)

B = np.array([[7, 8],
              [9, 10],
              [11, 12]])   # Shape: (3, 2)

print("A (2x3):\n", A)
print("B (3x2):\n", B)

# Matrix multiplication
C = A @ B  # Result shape: (2, 2)
print("\nMatrix multiplication A @ B:")
print("Shape:", C.shape)
print("Result:\n", C)
# Output: Shape: (2, 2)
# Output:
#  [[ 58  64]
#   [139 154]]

# Compare with np.dot() and np.matmul()
print("\nOther ways to do matrix multiplication:")
print("np.dot(A, B):\n", np.dot(A, B))
print("np.matmul(A, B):\n", np.matmul(A, B))
print("All give the same result!")
```

Broadcasting in NumPy

❑ The Concept

- Broadcasting allows NumPy to perform arithmetic operations on arrays with different shapes without creating copying data (unnecessary memory copies).
- It expands smaller arrays logically, not physically → extremely memory-efficient.
- **Efficient**
- **Memory-safe**
- **Fast (C-level execution)**

Broadcasting in NumPy (Cont.)

- **Broadcasting Compatibility Rule**
  - Two dimensions are compatible when:
    - They are equal, OR
    - One of them is 1
  - If all dimensions (compared from right → left) follow this rule, broadcasting is possible.
- **Examples**
  - **Compatible:**
    - **(3, 3) and (3,)**
    - **(4, 1, 7) and (1, 5, 7)**
    - **(2, 3) and (1, 3)**
  - **Not compatible:**
    - **(3, 3) and (4,)**

Broadcasting in NumPy (Cont.)

- **Mechanics & np.newaxis**
- **Step 1: Shapes & Alignment**
  - matrix.shape = (3, 3)
  - vector.shape = (3,) → 1D array

```
matrix: (3, 3)
vector:    (3)
```

- **NumPy right-aligns shapes for broadcasting**
  - Missing dimensions are treated as 1, so the vector becomes:

```
vector → (1, 3)
```

- **Step 2: Row-wise Addition (Works Automatically)**
- **Why it works:**
  - (1,3) broadcasts across rows
  - Each row receives [v0, v1, v2]
- **This is NumPy's default broadcasting for a 1D vector.**

Broadcasting in NumPy (Cont.)

- **Step 3: Column-wise Addition (Requires np.newaxis)**
  - matrix + vector[:, np.newaxis]
- **Shape transformation:**
  - (3,) → (3, 1)
- **Now the shapes are:**
  - matrix: (3, 3)
  - vector: (3, 1)
- **Broadcast happens along columns → works perfectly.**
- **A (3,) vector behaves as (1,3) → row-wise broadcasting**
- **For column-wise broadcasting, you must add a new axis**

Example 1 – Basic Broadcasting

```
matrix = np.array([[1, 2, 3],    Matrix (3x3):
                   [4, 5, 6],
                   [7, 8, 9]])
vector = np.array([10, 20, 30])  Vector (3,)

result = matrix + vector
print(result)
# Output:
# Matrix + Vector (vector broadcasts to each row):
# [[11 22 33]
#  [14 25 36]
#  [17 28 39]]
```

Example 2 – np.newaxis - The Dimension Tool

```python
arr = np.array([1, 2, 3])
print("Original array:", arr)
print("Original shape:", arr.shape)
# Output: Original array: [1 2 3]
# Output: Original shape: (3,)

# Method 1: Using reshape
arr_col = arr.reshape(3, 1)
print("\n1. Using reshape(3, 1):")
print(arr_col)
print("Shape:", arr_col.shape)
# Output:
# [[1]
#  [2]
#  [3]]
# Output: Shape: (3, 1)

# Method 2: Using np.newaxis (cleaner!)
arr_col_newaxis = arr[:, np.newaxis]
print("\n2. Using arr[:, np.newaxis]:")
print(arr_col_newaxis)
print("Shape:", arr_col_newaxis.shape)
# Output: Same as above!
```

```python
# Add a row dimension
arr_row = arr[np.newaxis, :]
print("\n3. Using arr[np.newaxis, :] (row vector):")
print(arr_row)
print("Shape:", arr_row.shape)
# Output:
# [[1 2 3]]
# Output: Shape: (1, 3)

# Add dimensions to make 3D
arr_3d = arr[np.newaxis, :, np.newaxis]
print("\n4. Using arr[np.newaxis, :, np.newaxis] (3D):")
print(arr_3d)
print("Shape:", arr_3d.shape)
# Output:
# [[[1]
#   [2]
#   [3]]]
# Output: Shape: (1, 3, 1)
```

Example 3 – Practical Examples with np.newaxis

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Method without newaxis (wrong)
print("\nWithout newaxis (element-wise, wrong for outer product):")
print("a * b =", a * b)  # Element-wise, shape (3,)
# Output: a * b = [ 4 10 18]

# Method with newaxis (correct outer product)
print("\nWith newaxis (correct outer product):")
outer = a[:, np.newaxis] * b[np.newaxis, :]
print("a[:, np.newaxis] * b[np.newaxis, :]:")
print(outer)
print("Shape:", outer.shape)
# Output:
# [[ 4  5  6]
#  [ 8 10 12]
#  [12 15 18]]
# Output: Shape: (3, 3)
```

Concatenation (Joining Arrays)

- **Definition**
  - np.concatenate joins multiple NumPy arrays into one array along a specified axis.
  - **It does not create a new axis**, arrays must already be aligned along **all other dimensions**.
- **The Concatenation Rule: To concatenate successfully:**
  - All dimensions must match **exactly**,
  - **Except** for the dimension specified by axis, which is the one being extended.

Concatenation (Cont.)

- **np.concatenate((a1, a2, ...), axis=0)**
  - a1, a2, ... → arrays to join
  - axis → which dimension should grow
- **Visual Logic**
  - **axis = 0 → Vertical stacking (Default)**
    - Adds rows
    - Increases height
  - **axis = 1 → Horizontal stacking**
    - Adds columns
    - Increases width

Example 1 - Basic 1D concatenation

```
a1 = np.array([1, 2, 3])
a2 = np.array([4, 5, 6])
result = np.concatenate((a1, a2))
print(f"a1: {a1}")
print(f"a2: {a2}")
print(f"Concatenated (axis=0 default): {result}")
# a1: [1 2 3]
# a2: [4 5 6]
# Concatenated (axis=0 default): [1 2 3 4 5 6]
```

Example 2 - 2D arrays with axis=0 (vertical stacking)

```
a3 = np.array([[1, 2, 3],
               [4, 5, 6]])
a4 = np.array([[7, 8, 9],
               [10, 11, 12]])
result_axis0 = np.concatenate((a3, a4), axis=0)
print(f"a3 shape: {a3.shape}, a4 shape: {a4.shape}")
print(f"a3:\n{a3}")
print(f"a4:\n{a4}")
print(f"Concatenated (axis=0 - adds rows):\n{result_axis0}")
print(f"Result shape: {result_axis0.shape}")
print(f"Row count increased: 2 + 2 = 4 rows")
# a3 shape: (2, 3), a4 shape: (2, 3)
# a3:
# [[1 2 3]
#  [4 5 6]]
# a4:
# [[ 7  8  9]
#  [10 11 12]]
# Concatenated (axis=0 - adds rows):
# [[ 1  2  3]
#  [ 4  5  6]
#  [ 7  8  9]
#  [10 11 12]]
# Result shape: (4, 3)
# Row count increased: 2 + 2 = 4 rows
```

Example 3 - 2D arrays with axis=1 (horizontal stacking)

```
result_axis1 = np.concatenate((a3, a4), axis=1)
print(f"Concatenated (axis=1 - adds columns):\n{result_axis1}")
print(f"Result shape: {result_axis1.shape}")
print(f"Column count increased: 3 + 3 = 6 columns")
# Concatenated (axis=1 - adds columns):
# [[ 1  2  3  7  8  9]
#  [ 4  5  6 10 11 12]]
# Result shape: (2, 6)
# Column count increased: 3 + 3 = 6 columns
```

Example 4 - Multiple arrays concatenation

```
a5 = np.array([[0, 0, 0]])
a6 = np.array([[1, 1, 1]])
a7 = np.array([[2, 2, 2]])
result_multi = np.concatenate((a5, a6, a7), axis=0)
print(f"a5: {a5}")
print(f"a6: {a6}")
print(f"a7: {a7}")
print(f"Concatenated all:\n{result_multi}")
# a5: [[0 0 0]]
# a6: [[1 1 1]]
# a7: [[2 2 2]]
# Concatenated all:
# [[0 0 0]
#  [1 1 1]
#  [2 2 2]]
```

Example 5 - The RULE - Dimensions must match

```
try:
    b1 = np.array([[1, 2], [3, 4]])  # Shape (2, 2)
    b2 = np.array([[5, 6, 7]])       # Shape (1, 3)
    # This will FAIL because dimensions don't match
    result_error = np.concatenate((b1, b2), axis=1)
except ValueError as e:
    print(f"ERROR when axis=1: {e}")
    # ERROR when axis=1: all the input array dimensions except for the concatenation axis must match
exactly, but along dimension 0, the array at index 0 has size 2 and the array at index 1 has size 1
```

Example 6 - The RULE - Working case with matching dimensions

```
b3 = np.array([[1, 2], [3, 4]])  # Shape (2, 2)
b4 = np.array([[5, 6], [7, 8]])  # Shape (2, 2)
result_correct = np.concatenate((b3, b4), axis=1)
print(f"b3:\n{b3}")
print(f"b4:\n{b4}")
print(f"Concatenated (axis=1):\n{result_correct}")
# b3:
# [[1 2]
#  [3 4]]
# b4:
# [[5 6]
#  [7 8]]
# Concatenated (axis=1):
# [[1 2 5 6]
#  [3 4 7 8]]
```

Stacking in NumPy (Creating New Dimensions)

❑ **Concept:**

- Stacking joins arrays along a new axis, increasing the dimensionality of the result.
- This differs from concatenation, which extends an existing axis without changing the number of dimensions.

1. **np.stack (General Stacking Function)**
   - **It does NOT merge rows or columns.**
   - Creates a new dimension in the output.
   - All input arrays must have **exactly** the same shape.
   - The axis parameter determines where the new dimension is inserted.

Stacking in NumPy (Cont.)

- **Convenience Stacking Functions**
  - **np.vstack – Vertical Stacking**
    - Treats inputs as rows stacked on top of each other.
    - Equivalent to creating a new axis and stacking along axis 0.
    - Often used to combine multiple 1D vectors into a 2D matrix.
  - **np.hstack – Horizontal Stacking**
    - Stacks inputs side by side, increasing the number of columns.
    - Equivalent to stacking along axis 1.
    - Useful for combining feature vectors or expanding matrices.

Stacking in NumPy (Cont.)

- **np.dstack – Depth Stacking**
  - Stacks arrays along axis 2 (depth).
  - Produces a 3-dimensional array.
  - **Common in image processing:**
    - Stacking R, G, B grayscale channels into an RGB image.
    - Working with multi-channel data.

- **Key Distinction**

Operation	Effect on Dimensions	When Used
Concatenation	Extends an existing axis	Adding rows or columns
Stacking	Creates a new axis	Building matrices, tensors, RGB images

Why Stacking Is Important

1. Combine Multiple Arrays into a Single Tensor

- Real-world data often comes in multiple related arrays:
  - Features of multiple samples
  - Different sensor readings
  - Color channels of an image
- Stacking allows you to group them into one structured array without losing the individual structure.
- **Example:** RGB image from three 2D arrays:

Why Stacking Is Important (Cont.)

2. Add a New Dimension

- stack creates a new axis, which is required for many ML frameworks.
- **Example:** A dataset with 10 images, each 28×28 pixels:
  - **10** → number of samples
  - **28×28** → height × width
  - This structure is needed to feed a neural network.

```
images = np.stack([img1, img2, ..., img10])
# shape: (10, 28, 28)
```

Why Stacking Is Important (Cont.)

3. Preserve Structure

- Unlike concatenate, stacking doesn't flatten arrays.
- Each array stays intact, just grouped along a new axis.
- Keeps your data organized and multidimensional.
- Each array is preserved as a separate "row" (or slice)

```
a = [1, 2, 3]
b = [4, 5, 6]
stacked = np.stack([a, b])
# [[1 2 3]
#  [4 5 6]]
```

Why Stacking Is Important (Cont.)

4. Essential for Machine Learning & Deep Learning

- Most ML models expect tensors, not flat arrays
- **Examples:**
  - 2D images → (height, width, channels)
  - Time series → (time_steps, features)
  - Batches → (batch_size, height, width, channels)
- Stacking helps convert raw data into tensors

What is Tensor?

- A **tensor** is a generalization of scalars, vectors, and matrices to any number of dimensions.
- It is simply an N-dimensional array of numbers.

Name	Meaning	Example Shape
Scalar	Single number	()
Vector	1D array	(3,)
Matrix	2D array	(3, 4)
Tensor	3D, 4D, 5D… array	(3, 4, 5) or (10, 3, 32, 32)

Example 1 - Basic stacking vs concatenation

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.array([7, 8, 9])

# Concatenation
concatenated = np.concatenate([a, b, c])
print(f"Arrays: a={a}, b={b}, c={c}")
print(f"np.concatenate([a, b, c]): {concatenated}")
# np.concatenate([a, b, c]): [1 2 3 4 5 6 7 8 9]

# Stacking
stacked = np.stack([a, b, c])
print(f"np.stack([a, b, c]):\n{stacked}")
print(f"Shape: {stacked.shape}")
# np.stack([a, b, c]):
# [[1 2 3]
#  [4 5 6]
#  [7 8 9]]
# Shape: (3, 3)
```

Example 2 - Stacking along different axes (2D arrays)

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
y = np.array([[7, 8, 9],
              [10, 11, 12]])

print(f"x shape: {x.shape}\n{x}")
print(f"y shape: {y.shape}\n{y}")
# x shape: (2, 3)
# [[1 2 3]
#  [4 5 6]]
# y shape: (2, 3)
# [[ 7  8  9]
#  [10 11 12]]

# Stack along new axis 0 (creates new outermost dimension)
stack_axis0 = np.stack([x, y], axis=0)
print(f"\nnp.stack([x, y], axis=0):")
print(f"Shape: {stack_axis0.shape}")
print(f"Array:\n{stack_axis0}")
# Shape: (2, 2, 3)
# Array:
# [[[ 1  2  3]
#   [ 4  5  6]]
#  [[ 7  8  9]
#   [10 11 12]]]
```

Meaning of axis=0“

- Create a new dimension at the front.
- So the new axis is placed before the existing (2, 3) dimensions.

(number_of_arrays , rows , columns)

- 2 → x and y
- 2 → rows
- 3 → columns

```
stack_axis0 =
[
  [[ 1  2  3]
   [ 4  5  6]],   ← x

  [[ 7  8  9]
   [10 11 12]]    ← y
]
```

Example 2 - Stacking along different axes (2D arrays)

```
stack_axis1 = np.stack([x, y], axis=1)
print(f"\nnp.stack([x, y], axis=1):")
print(f"Shape: {stack_axis1.shape}")
print(f"Array:\n{stack_axis1}")
# Shape: (2, 2, 3)
# Array:
# [[[ 1  2  3]
#   [ 7  8  9]]
#  [[ 4  5  6]
#   [10 11 12]]]
```

Axis 1 means: inserts a new axis at position 1.

(rows , number_of_arrays , columns)

"For each row, group the corresponding rows from all arrays. **So:**

- Row **0** of **x** is grouped with row **0** of **y**
- Row **1** of **x** is grouped with row **1** of **y**

```
stack_axis1 =
[
  [[ 1  2  3],
   [ 7  8  9]],

  [[ 4  5  6],
   [10 11 12]]
]
```

Example 3 - Convenience functions - vstack and hstack

```
p = np.array([1, 2, 3])
q = np.array([4, 5, 6])

# vstack (vertical stack) - adds rows
vstacked = np.vstack([p, q])
print(f"np.vstack([p, q]):\n{vstacked}")
print(f"Shape: {vstacked.shape}")
# np.vstack([p, q]):
# [[1 2 3]
#  [4 5 6]]
# Shape: (2, 3)

# hstack (horizontal stack) - adds columns
hstacked = np.hstack([p, q])
print(f"\nnp.hstack([p, q]): {hstacked}")
print(f"Shape: {hstacked.shape}")
# np.hstack([p, q]): [1 2 3 4 5 6]
# Shape: (6,)
```

```
# For 2D arrays
print(f"\nFor 2D arrays:")
print(f"np.vstack([x, y]):\n{np.vstack([x, y])}")
print(f"np.hstack([x, y]):\n{np.hstack([x, y])}")
# np.vstack([x, y]):
# [[ 1  2  3]
#  [ 4  5  6]
#  [ 7  8  9]
#  [10 11 12]]
# np.hstack([x, y]):
# [[ 1  2  3  7  8  9]
#  [ 4  5  6 10 11 12]]
```

np.vstack vs np.stack(axis = 0)

Feature	np.vstack	np.stack(axis=0)
Primary Goal	Stack vertically (Row-wise).	Create a **new** dimension.
Dimension Change	Keeps it 2D (or higher).	**Always** increases dimensions by 1.
1D input (3,)	**Result:** (2, 3) (2D)	**Result:** (2, 3) (2D)
2D input (1, 3)	**Result:** (2, 3) (2D)	**Result: (2, 1, 3) (3D)**

Example 3 - Convenience functions - dstack

```
red_channel = np.array([[255, 0, 0],
                        [0, 255, 0]])
green_channel = np.array([[0, 255, 0],
                          [0, 0, 255]])
blue_channel = np.array([[0, 0, 255],
                         [255, 0, 0]])
rgb_image = np.dstack([red_channel, green_channel, blue_channel])
print(f"\nnp.dstack([R, G, B]):")
print(f"Shape: {rgb_image.shape}  # (height, width, channels)")
print(f"RGB Image:\n{rgb_image}")
# Shape: (2, 3, 3)  # (height, width, channels)
# RGB Image:
# [[[255   0   0]
#   [  0 255   0]
#   [  0   0 255]]
#  [[  0   0 255]
#   [255   0   0]
#   [  0 255   0]]]

# Show pixel values
print(f"\nPixel at (0,0): {rgb_image[0, 0]}  # Red: [255, 0, 0]")
print(f"Pixel at (0,1): {rgb_image[0, 1]}  # Green: [0, 255, 0]")
print(f"Pixel at (0,2): {rgb_image[0, 2]}  # Blue: [0, 0, 255]")
# Pixel at (0,0): [255   0   0]  # Red: [255, 0, 0]
# Pixel at (0,1): [  0 255   0]  # Green: [0, 255, 0]
# Pixel at (0,2): [  0   0 255]  # Blue: [0, 0, 255]
```

Visual table of pixels for each channel:

Pixel	Red	Green	Blue
(0,0)	255	0	0
(0,1)	0	255	0
(0,2)	0	0	255
(1,0)	0	0	255
(1,1)	255	0	0
(1,2)	0	255	0

Each pixel is now a triplet [R, G, B]:

```
rgb_image =
[
  [[255, 0, 0],   [0, 255, 0],   [0, 0, 255]],
  [[0, 0, 255],   [255, 0, 0],   [0, 255, 0]]
]
```

Histograms, Binning & Missing Data Handling

❑ Histograms — np.histogram()

- Used to analyze data distribution by counting how many values fall into specified intervals (bins).
- **Produces:**
  - **Bin counts** (how many values per bin)
  - **Bin edges** (interval boundaries)

$$Bin\ Width = \frac{Max\ - Min}{Bin\ Counts}$$

Histograms, Binning & Missing Data Handling

❑ When to Use a Histogram

1. Exploratory Data Analysis (EDA)

- Before applying statistical models or machine learning, you need to see how your data is distributed.
- **Example:** Age distribution of customers, exam scores, or income levels.

2. Detect Skewness or Symmetry

- Histogram shapes reveal whether data is normally distributed, skewed, or uniform.
- Helps decide which statistical methods or transformations to use.

3. Identify Outliers

- Extreme values become apparent as bins with very low counts or isolated bars.

Histograms, Binning & Missing Data Handling

❑ When to Use a Histogram

4. Compare Multiple Groups

- ➢ Overlay histograms to compare distributions across categories.
- ➢ **Example:** Compare exam scores of two classes.

5. Frequency Analysis

- ➢ Understand how often values fall in certain ranges (bins).
- ➢ Useful in finance, quality control, or risk analysis.

6. Preprocessing for Machine Learning

- ➢ Helps decide if you need normalization, scaling, or binning.
- ➢ **Example:** Age bins (18–25, 26–35, etc.) for categorical features.

Example 1

```
data = [1, 2, 2, 3, 4, 5, 5, 5, 6]
counts, bin_edges = np.histogram(data, bins=3)
```

- **Step 1: Determine bin edges**
  - bins=3 → divide the range of the data into 3 equal-width intervals.
  - Minimum value = 1, Maximum value = 6
  - Width of each bin: $\boldsymbol{bin\ width} = \frac{6-1}{3} = \mathbf{1.6667}$
  - So bins are:
    - **Bin 1:** 1.0 → 2.6667
    - **Bin 2:** 2.6667 → 4.3333
    - **Bin 3:** 4.3333 → 6.0

Example 1 (Cont.)

- **Step 2: Count values in each bin**
  - **Bin 1: 1.0 ≤ x < 2.6667**
    - Values from data: 1, 2, 2 → **3** values
  - **Bin 2: 2.6667 ≤ x < 4.3333**
    - Values from data: 3 → **1** value
  - **Bin 3: 4.3333 ≤ x ≤ 6.0**
    - Values from data: 4, 5, 5, 5, 6 → **5** values
  - **Note:** The rightmost bin includes the right edge by default in NumPy. All other bins are half-open intervals [left, right). That's why 6 falls into bin 3, not outside.

Example 1 (Cont.)

Bin	Interval	Values	Count
1	[1.0, 2.6667)	1, 2, 2	3
2	[2.6667, 4.3333)	3	1
3	[4.3333, 6.0]	4, 5, 5, 5, 6	5

Example 2

```
data = np.array([1.2, 2.5, 3.1, 4.7, 5.2, 1.8, 2.9, 3.6, 4.1, 5.8,
                 2.1, 3.3, 4.5, 5.1, 1.5, 2.7, 3.9, 4.3, 5.6, 1.9])

print("Data:", data)
print("Data shape:", data.shape)
# Output: Data: [1.2 2.5 3.1 4.7 5.2 1.8 2.9 3.6 4.1 5.8 2.1 3.3 4.5 5.1 1.5 2.7 3.9 4.3 5.6 1.9]
# Output: Data shape: (20,)

# Basic histogram with 5 bins
counts, bin_edges = np.histogram(data, bins=5)
print("\nHistogram with 5 bins:")
print("Bin counts:", counts)
print("Bin edges:", bin_edges)
# Output:
# Bin counts: [4 6 5 3 2]
# Bin edges: [1.2  2.12 3.04 3.96 4.88 5.8 ]

print("\nInterpretation:")
print(f"- {counts[0]} values in range [{bin_edges[0]:.2f}, {bin_edges[1]:.2f})")
print(f"- {counts[1]} values in range [{bin_edges[1]:.2f}, {bin_edges[2]:.2f})")
print(f"- {counts[2]} values in range [{bin_edges[2]:.2f}, {bin_edges[3]:.2f})")
print(f"- {counts[3]} values in range [{bin_edges[3]:.2f}, {bin_edges[4]:.2f})")
print(f"- {counts[4]} values in range [{bin_edges[4]:.2f}, {bin_edges[5]:.2f}]")
# Output:
# - 4 values in range [1.20, 2.12)
# - 6 values in range [2.12, 3.04)
# - 5 values in range [3.04, 3.96)
# - 3 values in range [3.96, 4.88)
# - 2 values in range [4.88, 5.80]
```

Example 3 - Different Ways to Specify Bins

```
data = np.array([1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 5, 6, 7, 8, 9, 10])

print("Data:", data)

# Method 1: Specify number of bins
counts1, edges1 = np.histogram(data, bins=5)
print("\n1. bins=5 (auto edges):")
print("   Counts:", counts1)
print("   Edges:", edges1)
# Output:
# Counts: [4 4 4 3 2]
# Edges: [ 1.   2.8  4.6  6.4  8.2 10. ]

# Method 2: Specify bin edges explicitly
counts2, edges2 = np.histogram(data, bins=[1, 3, 6, 10])
print("\n2. bins=[1, 3, 6, 10] (custom edges):")
print("   Counts:", counts2)
print("   Edges:", edges2)
# Output:
# Counts: [3 9 5]
# Edges: [ 1  3  6 10]
```

Here we defined edges [1, 3 , 6 , 10]

So, we have 3 Bins

Bin 1: $1 \leq x \leq 3$ **count = 3**

Bin 2: $3 \leq x \leq 6$ **count = 9**

Bin 3: $6 \leq x \leq 10$ **count = 5**

Example 4 - Different Ways to Specify Bins (Cont.)

```
data = np.array([1, 2, 2, 3, 3, 3, 4, 4, 5, 5, 5, 5, 6, 7, 8, 9, 10])
# Method 3: Specify number of bins and range
counts3, edges3 = np.histogram(data, bins=4, range=(2, 8))
print("\n3. bins=4, range=(2, 8):")
print("   Counts:", counts3)
print("   Edges:", edges3)
print("   Note: Only values between 2 and 8 are counted!")
# Output:
# Counts: [3 3 2 1]
# Edges: [2.  3.5 5.  6.5 8. ]

# Method 4: Bin width instead of count
bin_width = 2.5
bins4 = np.arange(data.min(), data.max() + bin_width, bin_width)
counts4, edges4 = np.histogram(data, bins=bins4)
print("\n4. Fixed bin width (2.5):")
print("   Bin edges:", bins4)
print("   Counts:", counts4)
# Output:
# Bin edges: [ 1.   3.5  6.   8.5 11. ]
# Counts: [6 6 3 2]
```

Binning & Digitizing — np.digitize()

- **np.digitize() — Discretizing Continuous Data**
- **Purpose**
  - **np.digitize() converts continuous numerical values into discrete bin indices based on specified boundaries.**
- **Why We Need It**
  - np.digitize() is commonly used for:
  - Grouping numeric values into ranges
  - Feature discretization in Machine Learning
  - Categorizing data using thresholds
  - Rule-based classification

Binning & Digitizing — np.digitize() Cont.

❑ **Syntax: np.digitize(x, bins, right=False)**

- **x:** Array of values to bin
- **bins:** Array of monotonically increasing bin edges
- **right:** Whether intervals include the right edge
  - **False (default):** intervals are [bin[i-1], bin[i])
  - **True:** intervals are (bin[i-1], bin[i]]

```
bins = [0, 10, 20, 30]
Value 5 → Bin 1  (0 ≤ 5 < 10)
Value 15 → Bin 2 (10 ≤ 15 < 20)
Value 25 → Bin 3 (20 ≤ 25 < 30)
```

Example 1 – Basic digitization

```
values = np.array([1.2, 5.7, 12.3, 15.8, 21.5, 25.9, 30.1])
bins = [0, 10, 20, 30, 40]
indices = np.digitize(values, bins)
print(f"Values: {values}")
# Values: [ 1.2  5.7 12.3 15.8 21.5 25.9 30.1]
print(f"Bin indices: {indices}")
# Bin indices: [1 1 2 2 3 3 4]
```

Example 2 – Right vs left inclusive intervals

```
values = np.array([1, 10, 20, 30])
bins = [0, 10, 20, 30]
indices_left = np.digitize(values, bins, right=False)
indices_right = np.digitize(values, bins, right=True)
print(f"Values: {values}")
# Values: [ 1 10 20 30]
print(f"Left-inclusive indices: {indices_left}")
# Left-inclusive indices: [1 2 3 4]
print(f"Right-inclusive indices: {indices_right}")
# Right-inclusive indices: [1 1 2 3]
```

Example 3 – age Grouping

```
ages = np.array([17, 22, 25, 32, 38, 45, 51, 67, 72])
age_bins = [0, 18, 25, 35, 50, 65, 100]
age_groups = np.digitize(ages, age_bins)
group_labels = ['Under 18', '18-24', '25-34', '35-49', '50-64', '65+']
print(f"Ages: {ages}")
# Ages: [17 22 25 32 38 45 51 67 72]
print(f"Group indices: {age_groups}")
# Group indices: [1 2 3 3 4 4 5 6 6]
for i, age in enumerate(ages):
    print(f"Age {age}: {group_labels[age_groups[i]-1]}")
    # Age 17: Under 18
    # Age 22: 18-24
    # Age 25: 25-34
    # Age 32: 25-34
    # Age 38: 35-49
    # Age 45: 35-49
    # Age 51: 50-64
    # Age 67: 65+
    # Age 72: 65+
```

Example 4 – Test Score Grading

```
scores = np.array([45, 67, 82, 91, 58, 73, 89, 95, 61, 77])
grade_bins = [0, 60, 70, 80, 90, 100]
grade_indices = np.digitize(scores, grade_bins)
grade_letters = ['F', 'D', 'C', 'B', 'A']
print(f"Scores: {scores}")
# Scores: [45 67 82 91 58 73 89 95 61 77]
print(f"Grade indices: {grade_indices}")
# Grade indices: [1 2 3 4 1 3 4 5 2 3]
for score, idx in zip(scores, grade_indices):
    print(f"Score {score:3d}: {grade_letters[idx-1]}")
    # Score  45: F
    # Score  67: D
    # Score  82: B
    # Score  91: A
    # Score  58: F
    # Score  73: C
    # Score  89: B
    # Score  95: A
    # Score  61: D
    # Score  77: C
```

- **zip()** pairs each score with its bin index
- **idx - 1** is used because:
  - **np.digitize()** returns 1-based indices
  - **Python lists** are 0-based

Example 5 – Creating ML features

```python
np.random.seed(20)
house_prices = np.random.normal(300000, 100000, 10)
price_bins = [0, 150000, 300000, 450000, 600000, np.inf]
price_categories = np.digitize(house_prices, price_bins)
print(f"House prices: {house_prices.astype(int)}")
# House prices: [234768 218939 474512 370990 283755 260432 431240 354965 308779 220922]
print(f"Price categories: {price_categories}")
# Price categories: [2 2 4 3 2 2 4 3 3 2]

# Show bin ranges and mapping
print("\nBin ranges and classification:")
for i, (price, category) in enumerate(zip(house_prices, price_categories)):
    print(f"House {i+1}: ${price:,.0f} → Category {category}")
    # House 1: $234,768 → Category 2
    # House 2: $218,939 → Category 2
    # House 3: $474,512 → Category 4
    # House 4: $370,990 → Category 3
    # House 5: $283,755 → Category 2
    # House 6: $260,432 → Category 2
    # House 7: $431,240 → Category 4
    # House 8: $354,965 → Category 3
    # House 9: $308,779 → Category 3
    # House 10: $220,922 → Category 2
```

np.random.seed(10)
This creates 10 random values drawn from a normal (Gaussian) distribution.
Parameters:

- 300000 → mean (average house price)
- 100000 → standard deviation (price variability)
- 10 → number of samples

Result: prices centered around $300,000, with realistic variation.

- Ensures **reproducibility**
- Every time this code runs, it produces the same random values
- **Essential for:**
  - Debugging
  - Teaching
  - Scientific experiments

Structured Arrays (The C-Struct of NumPy)

- **The Problem**
  - Standard NumPy arrays are homogeneous: every element must have the same data type (all int, all float, all bool, etc.).
  - Real-world datasets (like spreadsheets, CSVs, SQL tables) contain mixed types, for example:
    - Name → string
    - Age → integer
    - Salary → float
  - Python lists or dictionaries can store mixed types, but:
    - They are slow,
    - Consume more memory,
    - Have poor cache locality.

Structured Arrays (The C-Struct of NumPy)

❑ The Solution: Structured Arrays

- NumPy provides structured arrays that allow fields of different data types inside a single array element.
- Each element acts like a C struct with named fields:
  - **Example fields:** "name", "age", "salary“
- Enables column-wise access, efficient filtering, and compatibility with binary data formats.

Structured Arrays (The C-Struct of NumPy)

- **Memory Layout**
  - Data is stored in a contiguous, binary-packed format, similar to:
    - C structs
    - Database records
    - Memory-mapped files
  - **Benefits:**
    - Better cache locality than Python dictionaries or objects
    - Lower overhead
  - **Ideal for:**
    - Large datasets
    - Reading binary files
    - Interfacing with C/C++ code
    - Scientific data pipelines

Structured Arrays (The C-Struct of NumPy)

❑ Key Idea

➢ Structured arrays let NumPy behave like a lightweight in-memory database table:

- Mixed types
- Fast vectorized operations
- Efficient memory layout
- Still behaves like a NumPy array

How They Work & Why They Matter

❑ How Structured Arrays Are Organized

- A structured array element contains multiple named fields, each with its own data type.
- Fields are stored in a fixed, contiguous memory block, in a layout similar to:
  - C structs
  - Packed binary records
- Because fields are laid out consecutively in memory, NumPy can:
  - Read each column efficiently
  - Vectorize computations directly over specific fields
  - Avoid Python object overhead

Advantages Over Python Objects

1. **Performance**
   - Python objects (dicts, classes) store each attribute separately in memory.
   - Structured arrays store everything together, boosting:
     - CPU cache usage
     - Memory throughput
     - Vectorized operation performance
2. **Low Memory Overhead**
   - Python objects require:
     - Pointers
     - Headers
     - Reference counters
   - Structured arrays use raw bytes only → much smaller footprint.

Comparison: Structured Arrays vs Alternatives

Feature	Structured Arrays	Python Object List	Pandas DataFrame
Mixed data types	Yes	Yes	Yes
Memory efficiency	High	Low	Medium
Speed (vectorized ops)	High	Very low	Medium-high
Column access	Fast	Slow	Fast
Interfacing with C	Excellent	Poor	Limited
Ideal use case	Raw, typed, performant binary-like data	Small datasets, flexible objects	Data analysis with indexing, statistics

Example 1: Basic structured array definition

```
# Define dtype with field names and data types
dtype = [('name', 'U10'),    # Unicode string up to 10 characters
         ('age', 'i4'),      # 4-byte integer
         ('salary', 'f8')]   # 8-byte float (double)

# Create structured array
employees = np.array([('Alice', 25, 55000.0),
                      ('Bob', 30, 72000.0),
                      ('Charlie', 35, 85000.0)], dtype=dtype)

print(f"Employees structured array:\n{employees}")
print(f"Shape: {employees.shape}, dtype: {employees.dtype}")
print(f"Memory layout: {employees.data}")
# Employees structured array:
# [('Alice', 25, 55000.) ('Bob', 30, 72000.) ('Charlie', 35, 85000.)]
# Shape: (3,), dtype: [('name', '<U10'), ('age', '<i4'), ('salary', '<f8')]
# Memory layout: <memory at 0x...>

# Access fields by name
print(f"\nField access:")
print(f"All names: {employees['name']}")
# All names: ['Alice' 'Bob' 'Charlie']
print(f"All ages: {employees['age']}")
# All ages: [25 30 35]
print(f"All salaries: {employees['salary']}")
# All salaries: [55000. 72000. 85000.]
```

```
# Access individual records
print(f"\nIndividual records:")
print(f"First employee: {employees[0]}")
print(f"Name of first: {employees[0]['name']}")
# First employee: ('Alice', 25, 55000.)
# Name of first: Alice
```

Example 2 - Creating with np.zeros and filling

```
# Create empty structured array
students = np.zeros(3, dtype=[('id', 'i4'), ('grade', 'f4'), ('passed', '?')])
print(f"Empty students array:\n{students}")
# Empty students array:
# [(0, 0., False) (0, 0., False) (0, 0., False)]

# Fill the array
students['id'] = [101, 102, 103]
students['grade'] = [85.5, 92.0, 78.5]
students['passed'] = [True, True, True]

print(f"\nFilled students array:\n{students}")
# Filled students array:
# [(101, 85.5,  True) (102, 92. ,  True) (103, 78.5,  True)]
```

Example 3 - Multi-dimensional structured arrays

```
# Create 2x2 structured array
data_2d = np.zeros((2, 2), dtype=[('x', 'f4'), ('y', 'f4'), ('value', 'i4')])
data_2d['x'] = [[1.0, 2.0], [3.0, 4.0]]
data_2d['y'] = [[5.0, 6.0], [7.0, 8.0]]
data_2d['value'] = [[10, 20], [30, 40]]

print(f"2D structured array shape: {data_2d.shape}")
print(f"Array:\n{data_2d}")
print(f"\nAccess x field:\n{data_2d['x']}")
# 2D structured array shape: (2, 2)
# Array:
# [[(1., 5., 10) (2., 6., 20)]
#  [(3., 7., 30) (4., 8., 40)]]
# Access x field:
# [[1. 2.]
#  [3. 4.]]
```

Example 4 - Operations on structured arrays

```
products = np.array([('Apple', 1.99, 50),
                     ('Banana', 0.99, 100),
                     ('Orange', 1.49, 75),
                     ('Grapes', 2.99, 25)],
                    dtype=[('name', 'U10'), ('price', 'f4'), ('stock', 'i4')])

print(f"Products:\n{products}")
# Products:
# [('Apple', 1.99, 50) ('Banana', 0.99, 100) ('Orange', 1.49, 75) ('Grapes', 2.99, 25)]

# Filtering
cheap_products = products[products['price'] < 2.0]
print(f"\nProducts under $2.00:\n{cheap_products}")
# Products under $2.00:
# [('Apple', 1.99, 50) ('Banana', 0.99, 100) ('Orange', 1.49, 75)]

# Sorting
sorted_by_price = np.sort(products, order='price')
print(f"\nProducts sorted by price:\n{sorted_by_price}")
# Products sorted by price:
# [('Banana', 0.99, 100) ('Orange', 1.49, 75) ('Apple', 1.99, 50) ('Grapes', 2.99, 25)]
```